

1. [Practice collection](#)
2. Write code to compare and explain `checkedException` vs `uncheckedException`
3. Can there be multiple finally blocks? Can there be multiple catch blocks? Write code to explain.
4. When both catch and finally return values, what will be the final result?
5. What is the difference between **throw** and **throws**?
6. Run the below three pieces codes, Noticed the printed exceptions. why do we put the Null/Runtime exception before Exception ?

```
public class Main {
    public static void main(String[] args) {
        int a = 0;
        int b = 3
        String s = null;
        try {
            System.out.println(b / a);
            System.out.println(s.equals("aa"));
            throw new RuntimeException();
        } catch (ArithmeticException e) {
            e.printStackTrace();
        } catch (NullPointerException e) {
            e.printStackTrace();
        } catch (RuntimeException e) {
            e.printStackTrace();
        } catch (Exception e) {
            e.getMessage();
        }

        System.out.println("End ...");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        int a = 0;
        int b = 3
        String s = null;
        try {
            // System.out.println(b / a);
            System.out.println(s.equals("aa"));
            throw new RuntimeException();
        } catch (ArithmeticException e) {
            e.printStackTrace();
        } catch (NullPointerException e) {
```

```

        e.printStackTrace();
    } catch (RuntimeException e) {
        e.printStackTrace();
    } catch (Exception e) {
        e.getMessage();
    }

    System.out.println("End ...");
}

}

public class Main {
    public static void main(String[] args) {
        int a = 0;
        int b = 3
        String s = null;
        try {
            // System.out.println(b / a);
            // System.out.println(s.equals("aa"));
            throw new RuntimeException();
        } catch (ArithmeticException e) {
            e.printStackTrace();
        } catch (NullPointerException e) {
            e.printStackTrace();
        } catch (RuntimeException e) {
            e.printStackTrace();
        } catch (Exception e) {
            e.getMessage();
        }

        System.out.println("End ...");
    }
}

```

7. What is **optional**? why do you use it? write an optional example to demo how it avoids NPE.
8. What are the types of design patterns in Java ? Name popular design patterns, particularly [Creational Patterns](#) and [Structural Patterns](#)
9. Implement `Singleton`, `Factory`, and `Builder` patterns, explain how to guarantee thread-safe in your `singleton` pattern implementation.
10. Explain **SOLID** Principles ? Further explain **Open-Closed Principle (OCP)** ?
11. Liskov's substitution principle states that if class B is a subtype of class A, then object of type A may be substituted with any object of type B. What does this actually mean? (from OA) choose your answer.
 1. It mean that if the object of type A can do something, the object of type B could also be able tp perform the same thing

2. It means that all the objects of type A could execute all the methods present in its subtype B
3. It means if a method is present in class A, it should also be present in class B so that the object of type B could substitute object of type A.
4. It means that for the class B to inherit class A, objects of type B and objects of type A must be same.

12. Watch design pattern video as below.

singleton: <https://www.bilibili.com/video/BV1Np4y1z7BU?p=22>

Factory: https://www.bilibili.com/video/BV1Np4y1z7BU?p=35&vd_source=310561eab1216a27f7accf859bf7f6d9

Builder: https://www.bilibili.com/video/BV1Np4y1z7BU?p=50&vd_source=310561eab1216a27f7accf859bf7f6d9

Publisher_Subscriber: https://www.bilibili.com/video/BV1Np4y1z7BU?p=114&vd_source=310561eab1216a27f7accf859bf7f6d9